

# Bookshop — Maven Master Roadmap, Explained

A phase-by-phase companion for finishing the bookshop and mastering Maven.

You are on day 3 of Maven. You already built a working single-module project with real tests, resource filtering, and a fat jar. This guide takes your own roadmap (Phases 1–9) and explains **every concept on it**: what it is, why it exists, when you will use it, and how to build it *in your bookshop* — with code taken from your actual files, not invented examples.

Rules of the road:

1. **Type everything yourself.** Copy-pasting teaches your clipboard, not you.
  2. **Run the command after every change.** Maven is learned in the terminal.
  3. **Break things on purpose** when the guide says so. The error message is the lesson.
  4. Each phase ends with a **checkpoint**: commands whose output proves you understood the phase.
-

# Chapter 0 — Where You Are Right Now (Audit)

I read your project. Here is the honest status against your roadmap:

| PHASE                                     | STATUS                                                                                       |
|-------------------------------------------|----------------------------------------------------------------------------------------------|
| 1 — Core application                      | <b>Done</b> (missing: remove/update — see below)                                             |
| 2 — Resources & configuration             | <b>Half done</b> — filtering on, but <code>App</code> never loads the file                   |
| 3 — Database (JDBC)                       | Not started                                                                                  |
| 4 — Profiles                              | Started ( <code>dev</code> exists) — needs <code>test</code> , <code>prod</code> , and a fix |
| 5 — Testing (integration, tags, Failsafe) | Unit tests done; the rest not started                                                        |
| 6 — Packaging                             | <b>Done</b> (shade works) — one file to understand                                           |
| 7 — Local repository                      | Not started                                                                                  |
| 8 — Multi-module                          | Not started                                                                                  |
| 9 — Advanced Maven                        | Not started                                                                                  |

## Two bugs you have *right now* — fix them first

**Bug 0.1 — the placeholder that never resolves.** Your `src/main/resources/shop.properties` says:

```
shop.name=${shop.name}
version=${project.version}
```

and `shop.name` is defined **only inside the `dev` profile**. So run this:

```
mvn clean process-resources
cat target/classes/shop.properties
```

Without `-Pdev` , the output literally contains `shop.name=${shop.name}` — Maven does **not** fail on an unknown property during filtering; it silently leaves the placeholder text in the file. Your app would one day print `Welcome to ${shop.name}` in production. This exact bug ships to real production systems all the time.

**The fix:** always give filtered properties a *default* in the main `<properties>` block, and let profiles *override* it:

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.release>17</maven.compiler.release>
  <junit.version>5.11.0</junit.version>
  <shop.name>My Bookshop</shop.name>    <!-- default for all builds -->
</properties>
```

Rule to remember: **profiles override, properties provide defaults.**

**Bug 0.2 — the same plugin declared twice.** Your `<plugins>` block declares `exec-maven-plugin` **twice**: once with the `<mainClass>` configuration, once with the `announce` execution. Maven merges duplicate declarations, but the behavior is confusing and real projects treat it as an error. One plugin = one `<plugin>` block; multiple jobs go in multiple `<execution>` entries inside it. Also, your first guide told you to delete the `announce` execution once the phase-binding idea clicked — do that now. The merged, clean version:

```
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>exec-maven-plugin</artifactId>
  <version>3.3.0</version>
  <configuration>
    <mainClass>com.example.App</mainClass>
  </configuration>
</plugin>
```

**Also do this today (30 seconds):** the project is still not a git repository. Phases 3 and 8 restructure your whole project — you want an undo button before that.

```
cd ~/Desktop/Projects/LearningMaven_Junit/bookshop
git init
printf 'target/\n.idea/\n*.iml\n' > .gitignore
git add -A && git commit -m "bookshop: end of phase 1-2"
```

### Checkpoint 0

☐

`mvn clean process-resources && cat target/classes/shop.properties` shows a real shop name *without* `-Pdev`.

☐

Only one `exec-maven-plugin` block remains in the pom.

☐

`git log` shows your first commit.

---

# Phase 1 — Core Application (mostly done — one gap)

You have `Book`, `BookShop`, `App`, and tests. Your roadmap says “CRUD operations” — you have **C**reate (`addBook`) and **R**ead (`findByAuthor`), but no **U**ppdate or **D**eleate. Add the missing two, because Phase 3 (the database) will need all four:

```
// In BookShop.java
public boolean removeByTitle(String title) {
    return books.removeIf(b -> b.getTitle().equals(title));
}

public Optional<Book> findByTitle(String title) {
    return books.stream()
        .filter(b -> b.getTitle().equals(title))
        .findFirst();
}
```

**Why `Optional` ?** A search can legitimately find nothing. Returning `null` forces every caller to remember a null-check; forgetting one is the classic `NullPointerException`. `Optional<Book>` makes “maybe absent” part of the method’s *signature* — the compiler reminds the caller. Use `Optional` for return values of finder methods; never for fields or parameters.

**A Java note while you’re in this file:** rename the field `Books` to `books` in `BookShop.java`. In Java, capitalized names are for *types*, lower-case for *variables*. Every Java developer reading `Books.add(b)` stumbles for a second. Small thing, strong convention.

Write a test for each new method before moving on (you know JUnit now:

```
assertTrue(shop.removeByTitle("Clean Code")) , and for the Optional:
assertTrue(shop.findByTitle("Nope").isEmpty()) ).
```

## Checkpoint 1

☐

`mvn test` runs at least 6 tests, all green.

☐

You can say out loud what C, R, U, D map to in `BookShop`.

# Phase 2 — Resources & Configuration (finish it)

## The concept: what “resources” really are

`src/main/resources` files are copied into `target/classes` during the `process-resources` phase, which means they end up **inside your jar**, on the *classpath* — the set of locations the JVM searches for classes and files. That is the whole trick: your code can load them the same way whether it runs from IntelliJ, from `mvn exec:java`, or from a fat jar on a server, because in all three cases the file is “on the classpath”.

**When to use resources:** any file your app needs at runtime that ships *with* the app — config defaults, seed data, SQL scripts, templates. **When not to:** files that differ per machine or contain secrets. Those stay *outside* the jar (Phase 4 shows how profiles help).

## Exercise 2.1 — actually load `shop.properties` in `App`

Your file is filtered and copied — but no code reads it. Fix that:

```
package com.example;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

public class App {
    public static void main(String[] args) throws IOException {
        Properties config = new Properties();
        try (InputStream in =
            App.class.getResourceAsStream("/shop.properties")) {
            if (in == null) {
                throw new IllegalStateException(
                    "shop.properties not found on classpath");
            }
            config.load(in);
        }

        System.out.println("Welcome to " + config.getProperty("shop.name")
            + " v" + config.getProperty("version"));

        BookShop bookShop = new BookShop();
        // ... your existing books ...
        System.out.println(bookShop.totalValue());
        System.out.println(bookShop.toJson());
    }
}
```

Three things to understand in that code, because you will write it in every Java project for the rest of your life:

- `getResourceAsStream("/shop.properties")` — the leading `/` means “from the classpath root”. Without it, Java looks *relative to the package* ( `com/example/shop.properties` ). Beginners lose hours to this slash. Root your paths.
- `try (...)` — try-with-resources. The stream is closed automatically even if loading throws. Anything that implements `AutoCloseable` (streams, and in Phase 3: database connections!) belongs in one of these.
- **The null check** — `getResourceAsStream` returns `null` (not an exception) when the file is missing. Fail loudly and early with a clear message; the alternative is a confusing `NullPointerException` two lines later.

Run it and watch the filtered value flow all the way through:

```
mvn compile exec:java           # Welcome to My Bookshop v1.0-SNAPSHOT
mvn compile exec:java -Pdev     # Welcome to My Dev Bookshop v1.0-SNAPSHOT
```

## Exercise 2.2 — seed data: `books.json`

Your shop hardcodes its books in `main`. Move them to a resource — this is the “seed data” pattern, and it finally uses Gson in *both* directions:

`src/main/resources/books.json`:

```
[
  { "title": "Clean Code",      "author": "Robert C. Martin", "price": 39.99 },
  { "title": "Effective Java",  "author": "Joshua Bloch",     "price": 45.50 },
  { "title": "Design Patterns", "author": "Erich Gamma",      "price": 55.00 }
]
```

Loading JSON into objects (deserialization) in `BookShop`:

```
public void loadFrom(InputStream json) {
    Gson gson = new Gson();
    Book[] loaded = gson.fromJson(
        new InputStreamReader(json, StandardCharsets.UTF_8), Book[].class);
    books.addAll(Arrays.asList(loaded));
}
```

And in `App`, replace the three `addBook` calls:

```
try (InputStream in = App.class.getResourceAsStream("/books.json")) {
    bookShop.loadFrom(in);
}
```

**One catch, worth understanding:** filtering is ON for *all* of `src/main/resources` in your pom. If a JSON file ever contained the text `${...}`, Maven would try to substitute it. The professional pattern is to filter only what needs filtering:

```
<resources>
  <resource>
    <directory>src/main/resources</directory>
    <filtering>true</filtering>
    <includes><include>*.properties</include></includes>
  </resource>
  <resource>
    <directory>src/main/resources</directory>
    <filtering>false</filtering>
    <excludes><exclude>*.properties</exclude></excludes>
  </resource>
</resources>
```

Read it as: “properties files get filtered; everything else is copied untouched.” **When to use filtering:** stamping build-time facts (version, build date, profile name) into config. **When not to:** anything a user or another program wrote — binary files especially can be corrupted by filtering.

## Checkpoint 2

☐

`mvn exec:java` greets you with the shop name and version.

☐

The books come from `books.json`, not from Java code.

☐

You can explain the difference between `src/main/resources` and `target/classes`, and *which phase* copies one to the other.

---

# Phase 3 — Database Integration (the big new skill)

Until now your books die when the JVM exits. A database makes them *persistent*. This phase teaches JDBC (Java's low-level database API), the DAO pattern, and two Maven concepts: **dependency scopes** and keeping config out of code.

## 3.0 Concept map — the pieces and why each exists

```
App
├─ BookShop (business logic – knows NOTHING about SQL)
│   └─ BookDAO (interface – "a place books live")
│       ├── InMemoryBookDAO (a List – what you have today)
│       └─ JdbcBookDAO (real database)
│           └─ ConnectionFactory (opens connections)
│               └─ DatabaseConfig (reads database.properties)
```

**Why so many layers?** Each one answers one question:

- **DAO (Data Access Object)** — an *interface* that describes storage operations ( `save` , `findByTitle` , `findAll` , `deleteByTitle` ) without saying *how* they happen. Your `BookShop` depends only on this interface. Result: you can swap MySQL for in-memory (fast tests! Phase 5) without touching business logic. This is the single most transferable pattern in this whole guide — Spring's repositories, which you'll meet in your real project, are DAOs with superpowers.
- **ConnectionFactory** — one place that knows how to open a database connection, so the URL/user/password code isn't repeated in every method.
- **DatabaseConfig** — reads `database.properties` from the classpath (Phase 2 skill, reused!). Config lives in files, not in `.java` files, because config changes per environment but code shouldn't.

## 3.1 The driver dependency — and your first *scope* decision

A **JDBC driver** is a jar that translates Java's standard `Connection` / `PreparedStatement` calls into one database's network protocol. MySQL's:

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
  <scope>runtime</scope>
</dependency>
```

**Why `<scope>runtime</scope>` ?** Walk through what scope means with the three dependencies your pom now has:

| DEPENDENCY        | SCOPE             | COMPILE<br>CLASSPATH | TEST CLASSPATH | RUNTIME /<br>PACKAGED |
|-------------------|-------------------|----------------------|----------------|-----------------------|
| gson              | compile (default) | yes                  | yes            | yes                   |
| junit-jupiter     | test              | no                   | yes            | no                    |
| mysql-connector-j | runtime           | <b>no</b>            | yes            | yes                   |

Your *code* never mentions a MySQL class — you write against `java.sql.Connection`, an interface from the JDK. The driver is only needed when the program *runs*. Declaring it `runtime` makes the compiler **stop you** if you accidentally import `com.mysql.*` — protecting the swappability the DAO pattern just bought you.

**Rule: drivers, logging backends, anything you use only through an interface → `runtime`.**

Run `mvn dependency:tree` after adding it and find the driver in the output.

## 3.2 The schema — teach the database about books

`src/main/resources/schema.sql`:

```
CREATE TABLE IF NOT EXISTS books (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  title   VARCHAR(200) NOT NULL,
  author  VARCHAR(120) NOT NULL,
  price   DECIMAL(8,2) NOT NULL
);
```

Notes for a first schema: `AUTO_INCREMENT PRIMARY KEY` gives every row a unique id you didn't have to invent; `DECIMAL(8,2)` — not `DOUBLE` — for money, because binary doubles cannot represent 0.10 exactly (your JUnit delta `0.001` was already a hint of this!); `NOT NULL` makes the database enforce what your constructor already assumes.

Set up the database once (install MySQL or MariaDB with your package manager first):

```
sudo mysql
CREATE DATABASE bookshop;
CREATE USER 'shopuser'@'localhost' IDENTIFIED BY 'shoppass';
GRANT ALL PRIVILEGES ON bookshop.* TO 'shopuser'@'localhost';
```

## 3.3 Config: `database.properties` + `DatabaseConfig`

`src/main/resources/database.properties`:

```
db.url=jdbc:mysql://localhost:3306/bookshop
db.user=shopuser
db.password=shoppass
```

`DatabaseConfig.java` — Phase 2 code, new file:

```

package com.example.db;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

public final class DatabaseConfig {
    private final Properties props = new Properties();

    public DatabaseConfig() {
        try (InputStream in = DatabaseConfig.class
            .getResourceAsStream("/database.properties")) {
            if (in == null) {
                throw new IllegalStateException(
                    "database.properties not found on classpath");
            }
            props.load(in);
        } catch (IOException e) {
            throw new IllegalStateException(
                "Could not read database.properties", e);
        }
    }

    public String url()      { return props.getProperty("db.url"); }
    public String user()     { return props.getProperty("db.user"); }
    public String password() { return props.getProperty("db.password"); }
}

```

ConnectionFactory.java:

```

package com.example.db;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public final class ConnectionFactory {
    private final DatabaseConfig config;

    public ConnectionFactory(DatabaseConfig config) {
        this.config = config;
    }

    public Connection open() throws SQLException {
        return DriverManager.getConnection(
            config.url(), config.user(), config.password());
    }
}

```

**Why a factory instead of calling `DriverManager` everywhere?** One reason now, one later. Now: the credentials wiring lives in exactly one place. Later: when you meet *connection pools* (HikariCP, and inside Spring), you will swap this class's internals and nothing else changes. A factory is a seam — a place designed for future change.

## 3.4 The DAO — interface first, then two implementations

`BookDAO.java` — the contract:

```

package com.example;

import java.util.List;
import java.util.Optional;

public interface BookDAO {
    void save(Book book); // C
    List<Book> findAll(); // R
    Optional<Book> findByTitle(String title); // R
    List<Book> findByAuthor(String author); // R
    boolean updatePrice(String title, double newPrice); // U
    boolean deleteByTitle(String title); // D
}

```

`InMemoryBookDAO.java` — your current `ArrayList` logic moves here almost unchanged. Ten minutes of cut-and-paste; it will become your *test* DAO.

`JdbcBookDAO.java` — the real one. Here are three of the six methods; the pattern repeats, so write the others yourself:

```

package com.example.db;

import com.example.Book;
import com.example.BookDAO;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class JdbcBookDAO implements BookDAO {
    private final ConnectionFactory factory;

    public JdbcBookDAO(ConnectionFactory factory) {
        this.factory = factory;
    }

    @Override
    public void save(Book book) {
        String sql = "INSERT INTO books (title, author, price) VALUES (?, ?, ?)";
        try (Connection con = factory.open();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, book.getTitle());
            ps.setString(2, book.getAuthor());
            ps.setDouble(3, book.getPrice());
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new RuntimeException("save failed for " + book.getTitle(), e);
        }
    }

    @Override
    public List<Book> findAll() {
        String sql = "SELECT title, author, price FROM books";
        List<Book> result = new ArrayList<>();
        try (Connection con = factory.open();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                result.add(new Book(rs.getString("title"),
                                    rs.getString("author"),
                                    rs.getDouble("price")));
            }
        } catch (SQLException e) {
            throw new RuntimeException("findAll failed", e);
        }
        return result;
    }

    @Override

```

```

public boolean deleteByTitle(String title) {
    String sql = "DELETE FROM books WHERE title = ?";
    try (Connection con = factory.open();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, title);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        throw new RuntimeException("delete failed for " + title, e);
    }
}
}

```

Concepts inside that code — each one is a habit, not a detail:

- **PreparedStatement with ? placeholders — always.** Never build SQL by concatenating strings (`"WHERE title = '" + title + "'"`). A title like `O'Reilly's Guide` breaks it, and a *malicious* title deletes your table — SQL injection, the #1 web vulnerability for two decades. The `?` mechanism sends values separately from the SQL, making injection impossible.
- **try-with-resources on Connection, PreparedStatement, ResultSet.** Database connections are scarce (a MySQL server allows ~150 by default). Leak them and your app freezes under load. Same `try (...)` you learned in Phase 2 — this is why it exists.
- **executeUpdate() returns the affected-row count** — that's how `deleteByTitle` knows whether the book existed. `executeQuery()` returns a `ResultSet` you walk with `rs.next()`.
- **Wrapping SQLException in RuntimeException.** `SQLException` is *checked*: without wrapping, every caller up the chain must declare `throws SQLException` — spreading JDBC knowledge into code that shouldn't know the database exists. Wrap at the boundary. (Later you can define `class DataAccessException extends RuntimeException` — Spring does exactly this.)

### 3.5 Rewire `BookShop` to use the DAO

`BookShop` stops owning a list and starts *delegating*:

```

public class BookShop {
    private final BookDAO dao;

    public BookShop(BookDAO dao) {           // dependency injection – by hand
        this.dao = dao;
    }

    public void addBook(Book b) { dao.save(b); }

    public double totalValue() {
        return dao.findAll().stream().mapToDouble(Book::getPrice).sum();
    }

    public List<Book> findByAuthor(String a) { return dao.findByAuthor(a); }

    public String toJson() {
        return new GsonBuilder().setPrettyPrinting().create()
            .toJson(dao.findAll());
    }
}

```

Passing the DAO in through the constructor is called **dependency injection** — the class doesn't create its collaborator, it *receives* it. You are doing manually what the Spring framework automates. Feeling this pain (the wiring in `App` below) is the best preparation for appreciating Spring later.

`App.java` becomes the *composition root* — the one place that decides which implementation runs:

```

DatabaseConfig config = new DatabaseConfig();
ConnectionFactory factory = new ConnectionFactory(config);
BookDAO dao = new JdbcBookDAO(factory);           // ← the ONLY line that
BookShop bookShop = new BookShop(dao);           // knows MySQL is involved

```

Swap that one line to `new InMemoryBookDAO()` and the whole app runs without a database. That flexibility is what Phases 4 and 5 will exploit.

### Checkpoint 3

☐

`mvn exec:java` inserts books and prints them back **from MySQL** (run it twice — books persist between runs now; you may want an `if empty then seed` guard).

☐

`sudo mysql -e "SELECT * FROM bookshop.books"` shows your rows.

☐

You can explain: why the driver is `runtime` scope, why `?` placeholders, why the DAO is an interface.

☐

`git commit` — this was a big phase.

# Phase 4 — Profiles (one build, many environments)

## The concept

Your app now has config ( `database.properties` ) that **must** differ between your laptop ( `dev` ), the test environment ( `test` ), and the real server ( `prod` ) — different URLs, users, passwords. A **profile** is a named, optional slice of pom that activates on demand and typically swaps property values. Combined with resource filtering (Phase 2), one `mvn package -Pprod` produces a prod-configured jar.

**When to use profiles:** environment differences (URLs, credentials, log levels), optional build steps (skip slow checks in dev). **When NOT to:** producing *functionally different* apps — that's a code smell; the same jar should ideally run anywhere, only *config* changing. (In your future real project, Spring's `application-{profile}.properties` does this at *runtime* — same idea, later binding.)

## Exercise 4.1 — parameterize `database.properties`

Replace hardcoded values with placeholders:

```
db.url=${db.url}
db.user=${db.user}
db.password=${db.password}
```

Defaults in the main `<properties>` (remember Bug 0.1 — defaults always):

```
<properties>
...
<db.url>jdbc:mysql://localhost:3306/bookshop</db.url>
<db.user>shopuser</db.user>
<db.password>shoppass</db.password>
</properties>
```

And three profiles:

```

<profiles>
  <profile>
    <id>dev</id>
    <activation>
      <activeByDefault>true</activeByDefault>
    </activation>
    <properties>
      <shop.name>My Dev Bookshop</shop.name>
    </properties>
  </profile>

  <profile>
    <id>test</id>
    <properties>
      <shop.name>Bookshop TEST</shop.name>
      <db.url>jdbc:mysql://localhost:3306/bookshop_test</db.url>
    </properties>
  </profile>

  <profile>
    <id>prod</id>
    <properties>
      <shop.name>The Bookshop</shop.name>
      <db.url>jdbc:mysql://prod-server:3306/bookshop</db.url>
      <db.user>prod_user</db.user>
      <!-- no password here! see 4.3 -->
    </properties>
  </profile>
</profiles>

```

Create the test database too, so `-Ptest` actually works:

```

sudo mysql -e "CREATE DATABASE bookshop_test;
GRANT ALL PRIVILEGES ON bookshop_test.* TO 'shopuser'@'localhost';"

```

## Exercise 4.2 — see it work, three ways

```

mvn help:active-profiles           # which profiles are on right now?
mvn clean process-resources -Ptest
cat target/classes/database.properties  # → bookshop_test URL

mvn clean process-resources -Pprod
cat target/classes/database.properties  # → prod-server URL

```

Things to notice:

- `-P` selects profiles; several can combine: `-Ptest,prod` (last property definition wins — try it and check which URL survives).
- `activeByDefault` on `dev` turns off the moment you pass **any** `-P` — a classic surprise. `help:active-profiles` is how you check reality instead of guessing.
- Profiles can also activate automatically — by OS, JDK, or a property ( `mvn ... -Denv=ci` ). File that away; explicit `-P` is clearer while learning.

## Exercise 4.3 — the credentials rule

Notice `prod` sets **no password**. Real credentials never go in the pom — the pom is committed to git, and git never forgets. The standard escape hatches, in the order you'll meet them in real life:

1. `~/.m2/settings.xml` — a per-machine, never-committed file that can also define profiles and properties. The build server has one with prod values; your laptop doesn't.
2. **Environment variables** — `${env.DB_PASSWORD}` in the pom reads the `DB_PASSWORD` env var. This is what your GitHub Actions build (Phase 9) will use, fed from repository *secrets*.

Try option 2 right now so it's not abstract: set `<db.password>${env.DB_PASSWORD}</db.password>` in the prod profile, run `DB_PASSWORD=supersecret mvn process-resources -Pprod`, and cat the result.

### Checkpoint 4

☐

Three profiles exist; `mvn help:active-profiles` shows `dev` by default.

☐

`target/classes/database.properties` provably differs between `-Ptest` and `-Pprod`.

☐

You can explain why passwords don't live in the pom, and name two places they *can* live.

---

# Phase 5 — Testing, Level 2 (unit vs integration)

## The concept: two kinds of tests, two plugins

Everything you've written so far is a **unit test**: fast, isolated, no network, no database — pure Java in, assertion out. A test that talks to a real MySQL is an **integration test**: slower, needs infrastructure, can fail for reasons that aren't bugs (DB down, wrong password). Mixing the two in one bucket ruins both — so Maven ships two test plugins:

|                 | SUREFIRE                | FAILSAFE                                            |
|-----------------|-------------------------|-----------------------------------------------------|
| Runs in phase   | <code>test</code>       | <code>integration-test</code> + <code>verify</code> |
| File pattern    | <code>*Test.java</code> | <code>*IT.java</code>                               |
| Meant for       | unit tests              | integration tests                                   |
| Fails the build | immediately             | in <code>verify</code> , after cleanup              |

The odd-sounding name “Failsafe” is the point: it does **not** fail the build during `integration-test`, so the `post-integration-test` phase can still tear down containers/servers; the failure is reported in `verify`.

## Exercise 5.1 — your DAO makes unit testing easy (this is the payoff)

`BookShopTest` should no longer touch MySQL — inject the in-memory DAO:

```
@BeforeEach
void setUp() {
    bookShop = new BookShop(new InMemoryBookDAO());
    bookShop.addBook(new Book("Clean Code", "Robert C. Martin", 39.99));
    // ...
}
```

Same assertions as before, still milliseconds-fast, zero infrastructure. *This* is why the DAO interface exists. If you had baked JDBC directly into `BookShop`, every test would need a database.

## Exercise 5.2 — a real integration test

`src/test/java/com/example/db/JdbcBookDAOIT.java` — note the **IT** suffix:

```

package com.example.db;

import com.example.Book;
import org.junit.jupiter.api.*;
import java.sql.Connection;
import java.sql.SQLException;
import static org.junit.jupiter.api.Assertions.*;

@Tag("integration")
class JdbcBookDAOIT {

    private ConnectionFactory factory;
    private JdbcBookDAO dao;

    @BeforeEach
    void setUp() throws SQLException {
        factory = new ConnectionFactory(new DatabaseConfig());
        dao = new JdbcBookDAO(factory);
        try (Connection con = factory.open()) {
            con.createStatement().execute("DELETE FROM books");
        }
    }

    @Test
    void savedBookCanBeFoundAgain() {
        dao.save(new Book("Refactoring", "Martin Fowler", 47.00));

        var found = dao.findByTitle("Refactoring");

        assertTrue(found.isPresent());
        assertEquals("Martin Fowler", found.get().getAuthor());
    }

    @Test
    void deleteReturnsFalseWhenBookAbsent() {
        assertFalse(dao.deleteByTitle("Ghost Book"));
    }
}

```

New ideas in there:

- **@Tag("integration")** — a JUnit 5 label. Plugins (and IDEs) can include/exclude by tag: `mvn test -Dgroups=integration` runs only tagged tests; `-DexcludedGroups=integration` skips them. Tags are how big teams slice suites: `slow`, `smoke`, `requires-network`.
- **Cleaning the table in @BeforeEach** — integration tests must not depend on leftover state from a previous run or from each other. Every test starts from a known world. (This is also why they run against `bookshop_test`, never the real DB — wire your `test` profile in.)

- **Test naming as documentation** — `deleteReturnsFalseWhenBookAbsent` tells you the contract without opening `JdbcBookDAO`.

## Exercise 5.3 — wire Failsafe

Surefire's pattern ( `*Test.java` ) already ignores `JdbcBookDAOIT` — verify that: `mvn test` should not run it. Then add Failsafe:

```
<plugin>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.3.0</version>
  <executions>
    <execution>
      <goals>
        <goal>integration-test</goal>
        <goal>verify</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

Now observe the split in action:

```
mvn test      # unit tests only – works with MySQL stopped!
mvn verify    # unit tests, THEN package, THEN integration tests
```

Try it with MySQL stopped ( `sudo systemctl stop mysql` ): `mvn test` stays green, `mvn verify` fails — exactly the separation you want. A teammate without MySQL can still build; CI runs the full `verify`.

### Checkpoint 5

☐

`mvn test` runs only unit tests (no DB needed).

☐

`mvn verify` runs `JdbcBookDAOIT` and its report lands in `target/failsafe-reports/`.

☐

You can explain the `*Test` vs `*IT` convention and what `@Tag` adds beyond it.

# Phase 6 — Packaging (you’ve done it — now *understand* it)

You already ship a working fat jar. This phase is short: three understandings to lock in, using tools, not faith.

## 6.1 — Look inside your jars

```
mvn clean package
ls -lh target/*.jar
jar tf target/bookshop-1.0-SNAPSHOT.jar | head -30
jar tf target/bookshop-1.0-SNAPSHOT.jar | grep -c gson
```

`jar tf` lists a jar’s contents (it’s just a zip). You should see your classes, your resources ( `shop.properties` , `books.json` — Phase 2 promised they’d be in here!), **and** `com/google/gson/...` classes — physical proof of what “fat jar” means. Extract the manifest too:

```
unzip -p target/bookshop-1.0-SNAPSHOT.jar META-INF/MANIFEST.MF
```

`Main-Class: com.example.App` is the line your jar-plugin `<archive>` config wrote — it’s how `java -jar` knows where `main` is.

## 6.2 — The mystery file: `dependency-reduced-pom.xml`

That file sitting in your project root was generated by the **shade plugin**. Why: if someone used your fat jar as a *dependency*, they’d get Gson twice — once bundled inside your jar, once as your declared transitive dependency. So shade writes an alternate pom with the bundled dependencies *removed*, and installs that one. Harmless — but it belongs in `.gitignore` (add it), and knowing what generated files are for beats being puzzled by them.

## 6.3 — Fat jar vs thin jar: when each

- **Fat (shaded) jar** — one file, contains everything, `java -jar` and done. Use for: CLI tools, simple services, anything you copy to a server. Your bookshop: correct choice.
- **Thin jar + dependencies elsewhere** — the jar-plugin default. Use for: libraries (consumers bring their own dependencies — shading a *library* causes version conflicts for its users), and app servers that provide dependencies.
- In your real project: Spring Boot has its own `spring-boot-maven-plugin` doing a smarter fat jar (nested jars, not merged). Same problem, same idea, different tool.

### Checkpoint 6

☐

You located Gson’s classes inside your fat jar with `jar tf`.

☐

You can say what `dependency-reduced-pom.xml` is for.



`java -jar target/bookshop-1.0-SNAPSHOT.jar` runs the full app (DB and all) from a single file.

---

# Phase 7 — The Local Repository (become your own dependency)

## The concept

Every dependency you've ever used was downloaded into `~/.m2/repository`, laid out as `groupId/artifactId/version/`. `mvn install` copies *your* artifact into that same structure. From that moment, `com.example:bookshop:1.0-SNAPSHOT` is a coordinate any project on your machine can declare — exactly like Gson. This is the moment Maven's "everything is an artifact with coordinates" worldview clicks.

## Exercise 7.1 — install and inspect

```
mvn clean install
ls ~/.m2/repository/com/example/bookshop/1.0-SNAPSHOT/
```

You'll find the jar, the pom, and bookkeeping files. Note the lifecycle logic of what just happened: `install` sits *after* `verify`, so your integration tests ran first — Maven refuses to install an artifact that fails its quality gates. That ordering is the lifecycle lesson.

## Exercise 7.2 — consume your own artifact

Make a scratch project *next to* bookshop (not inside!):

```
cd ~/Desktop/Projects/LearningMaven_Junit
mvn archetype:generate -DgroupId=com.example -DartifactId=shop-client \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.5 -DinteractiveMode=false
```

In `shop-client/pom.xml`:

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>bookshop</artifactId>
  <version>1.0-SNAPSHOT</version>
</dependency>
```

In its `App.java`:

```
BookShop shop = new BookShop(new InMemoryBookDAO());
shop.addBook(new Book("Domain-Driven Design", "Eric Evans", 62.00));
System.out.println(shop.toJson());
```

`mvn -f shop-client/pom.xml compile exec:java` — your bookshop classes, resolved from `~/m2` by coordinates. Also run `mvn dependency:tree` in shop-client: Gson appears *transitively* through bookshop, because bookshop's pom declares it. You are now on the producer side of dependency management.

**One SNAPSHOT gotcha to experience:** change something visible in bookshop's `toJson`, run `mvn install` in bookshop, rebuild shop-client — the change appears. That's the meaning of `-SNAPSHOT`: “re-check for newer builds of this version.” A release version ( `1.0` ) is immutable — cached forever. Phase 9 returns to this.

### Checkpoint 7

☐

`shop-client` compiles against bookshop from the local repo.

☐

`dependency:tree` in shop-client shows gson under bookshop.

☐

You can explain SNAPSHOT vs release in one sentence.

---

# Phase 8 — Multi-Module (the graduation project)

## The concept — and an honest “when to use”

A **multi-module** (a.k.a. *reactor*) build is one parent pom orchestrating several sub-projects that build together, in dependency order, versioned as one. **Why teams do it:** enforced boundaries — code in `core` physically *cannot* call JDBC if the JDBC dependency only exists in the `database` module; the compiler enforces your architecture. Also: shared version/ dependency management in one place, and reusable pieces (another team can depend on `bookshop-core` without dragging MySQL along).

**Honest counterpoint:** for a program this size, a single module is objectively fine. You are doing this to *learn the mechanics* — which is a great reason, because nearly every real Java codebase you’ll join is multi-module, and “how do the poms relate?” will be your first-week question there.

## Target structure

```
bookshop-parent/
├─ pom.xml                packaging=pom – the orchestra conductor
├─ bookshop-core/
│   ├─ pom.xml
│   └─ src/...            Book, BookDAO, InMemoryBookDAO, BookShop,
                           unit tests    (NO Gson, NO JDBC here)
├─ bookshop-database/
│   ├─ pom.xml            depends on: bookshop-core
│   └─ src/...            DatabaseConfig, ConnectionFactory,
                           JdbcBookDAO, database.properties, schema.sql,
                           JdbcBookDAOT
└─ bookshop-app/
    ├─ pom.xml            depends on: bookshop-core, bookshop-database
    └─ src/...            App, shop.properties, books.json, shade plugin
```

Design decision worth pausing on: **where does Gson go?** `toJson` lives in `BookShop` (core) today, which would drag Gson into core. Purists keep core dependency-free: move JSON work to the app module (a small `JsonPrinter` class). Do that — feeling *why* (“core stays clean”) is the architectural lesson of the whole phase.

## 8.1 — The parent pom

`bookshop-parent/pom.xml` :

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>bookshop-parent</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>pom</packaging>      <!-- ← builds no jar itself -->

    <modules>
        <module>bookshop-core</module>
        <module>bookshop-database</module>
        <module>bookshop-app</module>
    </modules>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>17</maven.compiler.release>
        <junit.version>5.11.0</junit.version>
        <gson.version>2.11.0</gson.version>
        <mysql.version>8.4.0</mysql.version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.junit</groupId>
                <artifactId>junit-bom</artifactId>
                <version>${junit.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
            <dependency>
                <groupId>com.google.code.gson</groupId>
                <artifactId>gson</artifactId>
                <version>${gson.version}</version>
            </dependency>
            <dependency>
                <groupId>com.mysql</groupId>
                <artifactId>mysql-connector-j</artifactId>
                <version>${mysql.version}</version>
            </dependency>
            <!-- modules can depend on each other version-free too -->
            <dependency>
                <groupId>com.example</groupId>
                <artifactId>bookshop-core</artifactId>
                <version>${project.version}</version>
            </dependency>
        </dependencies>
    </dependencyManagement>

```

```

        </dependency>
        <dependency>
            <groupId>com.example</groupId>
            <artifactId>bookshop-database</artifactId>
            <version>${project.version}</version>
        </dependency>
    </dependencies>
</dependencyManagement>

<build>
    <pluginManagement>
        <!-- your existing pluginManagement block moves here,
            plus failsafe 3.3.0 -->
    </pluginManagement>
</build>
</project>

```

Your original pom's management blocks **finally move to where they always belonged** — this is what `dependencyManagement` was *for*. The pattern: **parent = versions and policy; child = “I use this”, version-free.**

## 8.2 — A child pom

`bookshop-core/pom.xml` — notice everything that *isn't* here:

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>com.example</groupId>
    <artifactId>bookshop-parent</artifactId>
    <version>1.0-SNAPSHOT</version>
  </parent>

  <artifactId>bookshop-core</artifactId>  <!-- groupId & version inherited -->

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-params</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>

```

bookshop-database/pom.xml adds:

```

<dependency>
  <groupId>com.example</groupId>
  <artifactId>bookshop-core</artifactId>  <!-- inter-module dependency -->
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

```

bookshop-app/pom.xml adds core + database + gson, plus the exec, jar ( `mainClass` ), and shade plugin config from your current pom.

## 8.3 — Migration plan (do it in this order)

1. `git commit` the working single-module state. Seriously.

2. Create `bookshop-parent/` with the parent pom and three empty module folders each holding a child pom and `src/main/java`, `src/test/java`.
3. Move files: `Book`, `BookDAO`, `InMemoryBookDAO`, `BookShop`, `BookShopTest` → `core`. `db/*` classes, `database.properties`, `schema.sql`, `JdbcBookDAOIT` → `database`. `App`, `shop.properties`, `books.json` → `app`. Packages stay the same; only the module (and thus the jar) changes.
4. Move the `<profiles>` block to the parent (profiles inherit).
5. From `bookshop-parent/`: `mvn clean verify`, and read the **reactor summary**:

```
[INFO] Reactor Summary for bookshop-parent 1.0-SNAPSHOT:
[INFO] bookshop-parent ..... SUCCESS
[INFO] bookshop-core ..... SUCCESS
[INFO] bookshop-database ..... SUCCESS
[INFO] bookshop-app ..... SUCCESS
```

Maven computed that order from the dependencies — you never told it. That's the **reactor**. Useful flags once this works:

```
mvn -pl bookshop-core test           # build just one module
mvn -pl bookshop-app -am package     # ...and (-am) whatever it needs first
```

1. Prove the boundary is real: add `import java.sql.Connection;` -using code to a core class → still compiles (JDK class), but try using `JdbcBookDAO` from core → **compile error**, core doesn't see the database module. Your architecture is now compiler-enforced. Remove the experiment.

## Checkpoint 8

☐

`mvn clean verify` from the parent: 4× SUCCESS in reactor order.

☐

`java -jar bookshop-app/target/bookshop-app-1.0-SNAPSHOT.jar` works.

☐

Child poms contain **zero** version numbers for managed deps.

☐

You can explain reactor order and what `-pl` / `-am` do.

# Phase 9 — Advanced Maven (the professional toolkit)

Each item here is a “know what it is + do it once in bookshop” topic.

## 9.1 Maven Wrapper (`mvnw`)

**Problem:** teammate has Maven 3.6, you have 3.9 — “works on my machine.” **Fix:** commit a tiny script that downloads and uses a *pinned* Maven:

```
cd bookshop-parent
mvn wrapper:wrapper -Dmaven=3.9.9
./mvnw clean verify          # identical build for every machine & CI
```

From now on type `./mvnw` instead of `mvn` in this project — and in your real project, *always* set up the wrapper on day one. (Spring Boot generators include it automatically. Now you know what that file is.)

## 9.2 `dependency:tree` and exclusions

```
mvn dependency:tree
```

Read the tree: `gson` and `mysql` are *direct*; things indented under them are *transitive* — dependencies of your dependencies, resolved automatically. Two related tools:

- **Conflicts.** When two paths bring different versions of one library, Maven picks the *nearest* declaration (“nearest-wins”) — not the newest! `mvn dependency:tree -Dverbose` shows what was omitted and why. In real projects this explains 90% of `NoSuchMethodError` at runtime.
- **Exclusions.** When a dependency drags in something you don’t want (conflicting logging framework is the classic):

```
<dependency>
  <groupId>some.lib</groupId>
  <artifactId>some-lib</artifactId>
  <exclusions>
    <exclusion>
      <groupId>commons-logging</groupId>
      <artifactId>commons-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

- Also run `mvn dependency:analyze` on each module — it flags *used but undeclared* dependencies (you’re borrowing a transitive; declare it) and *declared but unused* ones (delete them).

## 9.3 Optional dependencies

`<optional>true</optional>` on a dependency means: “I use this, but projects depending on *me* don’t automatically get it.” A library that can integrate with Redis *if present* marks its Redis client optional; users who want that feature declare it themselves. **When you’ll care:** the day bookshop-core is a shared library. Until then, just recognize it in other people’s poms.

## 9.4 BOM — you already use one

Your `junit-bom` import is a **Bill of Materials**: a pom whose whole job is a `dependencyManagement` list of versions that are tested together. Importing it ( `<type>pom</type><scope>import</scope>` ) is why your JUnit dependencies need no `<version>`. Where you’ll meet BOMs again: `spring-boot-dependencies` manages *hundreds* of libraries this way — the reason Spring Boot poms look so bare. You could even write a `bookshop-bom` if outside teams consumed your modules — same mechanism.

## 9.5 SNAPSHOT vs Release, `install` VS `deploy`

The full mental model in four lines:

- `1.0-SNAPSHOT` = “moving development version” — repositories re-check for updates; every `install` overwrites.
- `1.0` (release) = immutable, forever. Re-releasing a changed `1.0` is a cardinal sin — caches worldwide would disagree about its contents.
- `install` = publish to **your machine** ( `~/ .m2` ).
- `deploy` = publish to a **shared remote repository** so teammates and CI get it. Needs a `<distributionManagement>` section + credentials in `settings.xml`.

The release flow you’ll see at work: develop on `1.0-SNAPSHOT` → release day: re-version to `1.0`, build, deploy, tag in git → immediately bump to `1.1-SNAPSHOT`. (The `maven-release-plugin` or the `versions-maven-plugin` automate the re-versioning: try `mvn versions:set -DnewVersion=1.1-SNAPSHOT` in `bookshop-parent` and watch it update every module pom in one command.)

## 9.6 Repository managers (Nexus / Artifactory)

Where does `deploy` deploy *to*? Companies run their own repository server — Sonatype **Nexus** or JFrog **Artifactory**. It plays three roles: (1) hosts the company’s private artifacts, (2) *proxies* + *caches* Maven Central so 200 developers don’t each download the internet, (3) becomes the single choke-point for security scanning. In your real project you’ll likely find a `<repositories>` or `settings.xml` mirror section pointing at one — now you’ll know what you’re looking at. Nothing to do in bookshop; recognizing it is the skill. (If curious: Nexus runs locally in Docker in ten minutes — a good weekend experiment, not a day-3 task.)

## 9.7 Archetypes

You’ve used one twice ( `maven-archetype-quickstart` ) — an **archetype** is just a project template. Real teams sometimes maintain their own (“our standard microservice layout”). Know the term, move on; modern Spring projects usually start from `start.spring.io` instead.

## 9.8 CI/CD with GitHub Actions

**The concept:** Continuous Integration = a neutral machine builds and tests *every push*, so “works on my machine” stops being an argument. This is where everything in this guide converges — wrapper (9.1), `verify` lifecycle (Phase 5), profiles (Phase 4).

Push your repo to GitHub, then create `.github/workflows/build.yml`:

```
name: build
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest

    services:
      mysql:
        image: mysql:8.4
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: bookshop_test
          MYSQL_USER: shopuser
          MYSQL_PASSWORD: shoppass
        ports: ["3306:3306"]
        options: >-
          --health-cmd="mysqladmin ping -h localhost"
          --health-interval=10s --health-retries=10

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
          cache: maven
      - name: Build & run all tests
        run: ./mvnw -B clean verify -Ptest
```

Read the file top to bottom and connect each line to something you built: the MySQL **service container** exists because your Failsafe tests need a database; the `-Ptest` profile points them at `bookshop_test`; `verify` runs the *whole* quality gate; `cache: maven` is `~/.m2` persisted between runs; `-B` (batch mode) silences the download progress spam in logs. When the green check appears on your commit — that’s CI, working.

### Checkpoint 9



`./mvnw -v` works on a fresh terminal (wrapper committed).



You ran `dependency:tree -Dverbose` and found at least one version conflict resolution (or proved there are none).

☐

You can define, in one sentence each: BOM, SNAPSHOT, deploy, Nexus, archetype.

☐

A GitHub Actions run of your project is green.

---

# Graduation — Mapping Bookshop to Your Real Project

Every skill you just built maps directly onto a modern (likely Spring) codebase. When you open your real project, this is the translation table:

| YOU BUILT BY HAND                                          | REAL PROJECT EQUIVALENT                                                         |
|------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>getResourceAsStream</code> + <code>Properties</code> | <code>application.properties</code> auto-loaded                                 |
| Constructor-injecting <code>BookDAO</code>                 | <code>@Autowired</code> / constructor injection — Spring wires it               |
| <code>BookDAO</code> interface + JDBC impl                 | Spring Data <code>Repository</code> — impl generated for you                    |
| <code>ConnectionFactory</code>                             | HikariCP connection pool, auto-configured                                       |
| Maven <code>-P</code> profiles + filtering                 | Spring runtime profiles ( <code>application-prod.properties</code> )            |
| Shade fat jar                                              | <code>spring-boot-maven-plugin</code> repackaged jar                            |
| Parent pom + BOM                                           | <code>spring-boot-starter-parent</code> / <code>spring-boot-dependencies</code> |
| <code>*IT</code> + Failsafe + MySQL service                | <code>@SpringBootTest</code> + <code>Testcontainers</code>                      |
| GitHub Actions <code>verify</code>                         | the same, verbatim                                                              |

You will recognize *all* of it — because you've now felt the problem each piece solves. That was the point of building it the hard way first.

## The final master checklist

☐

Bugs 0.1 and 0.2 fixed; project in git.

☐

Full CRUD with `Optional` finders, all unit-tested.

☐

Config and seed data load from the classpath.

☐

JDBC DAO with `PreparedStatement`s and `try-with-resources`; MySQL persists your books.

☐

`dev` / `test` / `prod` profiles; no secret in any committed file.

☐

`mvn test` = fast and DB-free; `mvn verify` = the full gate.

☐

One fat jar runs the whole app.

☐

Three modules, reactor-ordered, versions only in the parent.

☐

Wrapper committed, CI green.

When every box is checked, you haven't "done a tutorial" — you have built, tested, packaged, restructured, and continuously integrated a database- backed Java application with Maven, from an empty folder. That *is* the skill. Go start the real project.

---

# Appendix — Command Cheat Sheet (new commands from this guide)

| COMMAND                                        | WHAT IT DOES                                            |
|------------------------------------------------|---------------------------------------------------------|
| <code>mvn clean process-resources</code>       | run filtering only; inspect <code>target/classes</code> |
| <code>mvn help:active-profiles</code>          | show which profiles are on                              |
| <code>mvn package -Pprod</code>                | build with a profile                                    |
| <code>mvn test -Dgroups=integration</code>     | run only tests tagged <code>integration</code>          |
| <code>mvn verify</code>                        | full gate: unit + package + integration tests           |
| <code>mvn install</code>                       | publish to <code>~/.m2/repository</code>                |
| <code>mvn -pl bookshop-core test</code>        | build one module of a reactor                           |
| <code>mvn -pl bookshop-app -am package</code>  | one module + its prerequisites                          |
| <code>mvn dependency:tree -Dverbose</code>     | show conflict resolutions                               |
| <code>mvn dependency:analyze</code>            | find undeclared/unused dependencies                     |
| <code>mvn versions:set -DnewVersion=...</code> | re-version all modules at once                          |
| <code>mvn wrapper:wrapper -Dmaven=3.9.9</code> | add the Maven wrapper                                   |
| <code>jar tf target/app.jar</code>             | list what's inside a jar                                |
| <code>./mvnw -B clean verify</code>            | what CI runs                                            |